

An Adaptive Stability Governor for Real-Time Rescheduling Strategy Selection in Partially Flexible Job Shops

Rakshit Chaturvedi

CSBS

VNRVJIE

Hyderabad, India

rakshitcaturvedi09@gmail.com

Abstract— Production schedules, in practice, are not oracles. They are not aware of any machine disruptions or processing time deviations that could occur in the near future. When disruptions or deviations occur, the schedules are often forced to change, and the scheduler has to choose from two choices, to either locally repair the existing schedule or create a completely new schedule using full reoptimization. Current systems use advanced repair and reoptimization techniques, but lack a formal way to decide whether to locally repair, fully reoptimize, or do nothing based on the current state of schedule.

This paper introduces a decision layer that chooses between eight different strategies (consisting of multiple repair methods, do-nothing and fully reoptimization) based on the utility-stability ratio. The ratio uses an adaptive threshold that updates online using a regret-based learning rule. The decision layer monitors the state of current schedule passively, and only becomes active when either of the two trigger fires. The schedule state consists of the current executing schedule, current average slack ($RD3(t)$), accumulated deviation since original preschedule, current adaptive threshold, recent disruption frequency estimate, and current shop utilization. The primary trigger is the arrival of a disruption event E , and the secondary trigger being the accumulated deviation since the last disruption (or) trigger fire. The decision layer is evaluated on Fisher-Thompson, Lawrence, and Taillard benchmarks in three different levels of disruption severity and five different baselines, including static-threshold parameterizations and always-reoptimize policies.

Keywords—production; scheduling; schedule nervousness; online learning; utility-stability ratio; slack; accumulated deviation; disruption frequency; shop utilization; job shop scheduling; adaptive threshold

I. INTRODUCTION

The nature of production scheduling is to be optimistic. When a schedule is generated, it assumes that there will be no deviations or disruptions while the schedule is being executed. But in reality, a job shop rarely goes as planned, one of the machines could go down, there could be delay in processing,

The response to a disruption event requires a decision before it requires an algorithm. A scheduler facing a machine breakdown must first determine what class of response is warranted: absorb the disruption passively within available slack, apply a local repair that restores feasibility while preserving schedule continuity, or invoke

full reoptimization that recovers efficiency at the cost of imposing changes on all downstream operations. This is the strategy selection problem, and it is distinct from the optimization problem that each strategy type delegates to its solver. The repair algorithms and reoptimization methods available to a modern scheduling system are mature; the mechanism for choosing among them in real time, given the current state of the executing schedule, is not. Vieira et al. (2003) and Ouelhadj & Petrovic (2009) both identify this gap in their foundational surveys, while taxonomies of rescheduling strategies are well established, no formal, adaptive criterion for selecting among them has been proposed.

Two families of approaches have addressed strategy selection indirectly without resolving it. Fixed-weight heuristics, exemplified by the utility-stability framework of Cowling and Johansson (2002), select the strategy that maximizes $A \times U - B \times S$, where U is the objective-value improvement over passive absorption and S is the cost of schedule changes. This formulation correctly identifies the tradeoff structure of the problem, but Cowling and Johansson demonstrate through their own simulation experiments that the optimal A/B ratio depends strongly on the disruption event set – a finding that directly implies a fixed parameterization will be suboptimal across varying disruption environments and that an adaptive mechanism is required. The authors identify this limitation explicitly but do not provide a mechanism for addressing it. Machine-learning approaches, including deep reinforcement learning (DRL) systems that represent the current state of the art, learn selection policies from historical or simulated disruption data and can respond to disruption characteristics in ways that fixed rules cannot. However, DRL-based systems share three structural limitations: they provide no formal guarantees on switching decisions, requiring retraining rather than recalibration when the disruption environment changes; they cannot adapt online without retraining, as occurs when production mixes shift or seasonal patterns alter disruption frequencies; and their decisions are not interpretable for shop floor practitioners who must communicate and justify rescheduling actions to operations, suppliers, and customers. Larsen and Pranzo (2019) propose a controller-solver architecture that separates monitoring and decision logic from optimization – a structural advance – but their controller selects only between do-nothing and full reoptimization, leaving the strategy selection problem within that range unaddressed.

This paper proposes a decision layer that resolves the strategy selection problem through a formally derived, real-time, adaptive mechanism. The decision layer evaluates each candidate strategy using a normalized benefit-cost ratio $R(s) = U(s, E, t) / \text{StabilityCost}(s, E, t)$ and selects the least disruptive strategy whose ratio meets or exceeds an adaptive threshold $\theta(t)$, updated online after each disruption event via a regret-based learning rule. The governor monitors a composite state vector – including current schedule slack $RD3(t)$ from Leon, Wu, and Storer (1994), accumulated deviation $\Delta(t)$, disruption frequency, and shop utilization – and incorporates a secondary trigger that fires when accumulated deviation exceeds a bound, detecting gradual schedule degradation from sequences of individually minor disruptions that no single-event trigger can identify. The cost function integrates Leon et al.'s (1994) robustness-aware efficiency surrogate $Z(S)$ with Rangsaritratamee, Ferrel, and Kurz's (2004) temporally-weighted stability measure, normalized for dimensional consistency so that the single parameter r reflects a genuine preference between efficiency and stability rather than an artefact of scale.

The present work makes four contributions. First, it provides a formally derived, normalized utility-stability ratio for real-time strategy selection that is scale-invariant and directly interpretable as efficiency gain per unit of nervousness imposed. Second, it provides an adaptive threshold mechanism $\theta(t)$ updated by an online regret signal computed from observed execution outcomes, requiring no expert calibration and recalibrating automatically when the disruption environment changes. Third, it formalizes the secondary trigger based on accumulated deviation $\Delta(t)$, providing a mechanism for proactive schedule repair between discrete disruption events – a failure mode that purely event-driven systems cannot detect. Fourth, it specifies the monitoring interface requirements for deployment in environments with existing MES/SCADA infrastructure, with reproducible experimental code released alongside the paper.

II. LITERATURE REVIEW

This section traces the development of dynamic production scheduling from foundational taxonomic frameworks through stability-aware rescheduling and robustness-based schedule design, arriving at the specific gap that motivates the present work. The review is structured to build the argument that all necessary components of a formally grounded, adaptive rescheduling governor exist in the prior literature, but that no prior paper has assembled them into a single decision mechanism.

A. Dynamic Scheduling Frameworks

Production scheduling under disruption has been studied since the 1990s, generating a rich taxonomy of strategies and trigger policies. Vieira, Hermann, and Lin (2003) provide the foundational framework, classifying rescheduling strategies along three dimensions: scope (affected operations only versus full remaining schedule), sequence (preserve original ordering versus allow changes), and timing (immediate versus deferred response). Their taxonomy identifies three canonical strategy types – right-shift repair, which preserves sequence and shifts affected operations forward by the disruption duration; partial repair, which reschedules a subset of affected

operations, and complete rescheduling, which regenerates the entire remaining schedule – establishing the vocabulary that all subsequent work inherits.

Ouelhadj and Petrovic (2009) extend this taxonomy through a comprehensive survey of dynamic scheduling in manufacturing systems, identifying event-driven, periodic, and hybrid trigger policies and documenting the extensive landscape of repair heuristics proposed across the literature. Critically, their survey identifies the open problem that motivates the present work: while frameworks for categorizing rescheduling strategies are mature, no formal criterion exists for selecting among them dynamically given the current state of an executing schedule. Every approach reviewed either applies a fixed strategy regardless of disruption characteristics, or relies on domain-expert heuristics that require manual calibration and cannot adapt to changing conditions.

B. Scheduling Stability and the Efficiency-Stability Tradeoff

The tension between schedule efficiency and schedule stability was formalized by Cowling and Johansson (2002), who define utility U as the objective-value improvement achievable by acting versus doing nothing, and stability S as the cost of schedule changes measured by start-time and completion-time deviations. They propose selecting the strategy that maximizes $A \times U - B \times S$, where A and B are fixed weights set before execution. Their simulation experiments demonstrate that the optimal A/B ratio depends strongly on the disruption event set – a finding that directly implies a fixed parameterization cannot be optimal across varying conditions and that an adaptive mechanism is needed.

The empirical significance of the efficiency-stability tradeoff is established by Rangsaritratamee, Ferrel, and Kurz (2004), who study dynamic scheduling in a job shop under continuous arrivals and demonstrate that a bi-criteria approach incorporating both efficiency and stability improves stability by up to four percent with less than one percent degradation in efficiency. Their stability measure combines start-time deviation with a total deviation penalty $PF(\gamma) = 10/\sqrt{\gamma}$ that penalizes imminent changes more heavily than distant ones – a formulation that present work adopts as the stability component of the governor's composite cost function. Critically, Rangsaritratamee et al. (2004) use a fixed rescheduling interval and do not address the question of when to trigger rescheduling or which strategy to apply; their contribution to this is the bi-criteria objective formulation, not the selection mechanism.

The stability metrics that underpin both papers originate in Wu, Storer, and Chang (1993), who study single-machine rescheduling and define start-time deviation $D_0(\pi) = \sum |t'_i - t_i|$ and sequence deviation $D_\square(\pi) = \sum |t'_i - t'_j|$ as the field-standard measures of schedule nervousness. These measures quantify the disruption imposed on all planning activities – supplier orders, material deliveries, operator preparation – that depend on the published schedule, establishing that nervousness has real economic consequences beyond its direct impact on makespan.

C. Schedule Robustness and Repair Methods

Leon, Wu, and Storer (1994) establish the formal definition of schedule robustness as $(S) = r \times E[M(S)] + (1-r) \times E[\delta(S)]$, a linear combination of expected post-disturbance makespan and expected schedule

delay. By algebraic substitution this simplifies to $R(S) = r \times M_o(S) + E[\delta(S)]$, revealing that $E[\delta(S)]$ always enters with full weight and r controls only the additional contribution of the deterministic makespan – an asymmetry with implications for cost function design. For the practically important case of multiple disruptions across all machines, Leon et al. demonstrate that $E[\delta(S)]$ is intractable and develop surrogate measures, of which $RD3(S) = \sum_{i \in N_f} s_{li} / |N_f|$ – the average slack across operations on fallible machines – is the most computationally efficient. Their GAROB genetic algorithm embeds the surrogate fitness function $Z(S) = r \times M_o(S) + (1-2r) \times RD3(S)$ and empirically identifies $r=0.85$ as the value that best balances post-disturbance makespan and delay. The present work adopts $Z(S)$ as the efficiency component of the governor's cost function and $RD3(t)$ as the primary real-time signal of remaining schedule absorption capacity.

Bean, Birge, Mittenthal, and Noon (1991) establish match-up scheduling as the canonical local repair strategy, demonstrating through theoretical convergence results grounded in economic turnpike theory and empirical validation on real automotive manufacturing data that reconnecting a disrupted schedule to its baseline at a computed future matchup point T consistently outperforms both rigid right-shift and full dynamic rescheduling. Their MUSA algorithm implements this strategy: it iteratively extends T until the resequencing cost on disrupted machines falls below a threshold EPS, accepting the result as the repair solution. The EPS threshold is the direct structural precursor to the governor's adaptive threshold $\theta(t)$. Where Bean et al. treat EPS as a fixed parameter requiring expert calibration, the present work provides the missing mechanism for setting it adaptively through online regret minimization. The MUSA algorithm is incorporated as the match-up repair branch (MU) in the governor's eight-strategy action space.

D. Current State of the Art and the Remaining Gap

The most recent generation of dynamic scheduling research applies deep reinforcement learning (DRL) to learn rescheduling policies from historical or simulated disruption data. These approaches represent a genuine advance over static parameterization – a trained agent can respond to disruption characteristics in ways that fixed rules cannot. However, DRL-based rescheduling systems share three limitations. First, learned policies provide no formal guarantees on switching decisions. Second, DRL policies require retraining when the disruption environment changes, as occurs when production mixes shift, new machines are installed, or seasonal demand patterns alter disruption frequencies. Third, DRL policy decisions are not interpretable for shop floor practitioners, who must trust and communicate scheduling decisions to operators, suppliers, and customers. A governor that selects strategies based on an explicit, interpretable ratio $R(s)=U(s)/\text{StabilityCost}(s)$ compared against a documented threshold $\theta(t)$ provides a traceable rationale for every decision.

Beyond machine learning, Larsen and Pranzo (2019) propose a controller-solver framework for dynamic rescheduling that separates the monitoring and decision functions from the optimization function – structurally similar to the present work. Their controller decides when to invoke the solver but not which strategy to apply; the strategy set is implicitly limited to do-nothing versus full

reoptimization. The present work generalizes this to an eight-strategy action space and provides a formally derived selection criterion rather than a heuristic trigger rule.

III. METHODOLOGY

This section presents the formal definition of the rescheduling governor, the scheduling environment in which it operates, and the complete specification of its decision mechanism.

A. Problem Formulation

Consider a partially flexible job shop scheduling environment with m machines and n jobs. Each job $j \in J$ consists of an ordered sequence of operations that must be processed in a fixed routing order. Each operation $i \in O$ has an eligible machine set $M_{\text{eligible}}(i) \subseteq M$ and a machine-dependent processing time p_{ik} for each $k \in M_{\text{eligible}}(i)$. In standard job shop case, $|M_{\text{eligible}}(i)| = 1$ for all operations; in the flexible extension, a fraction p_{flex} of operations has at least one alternative eligible machine.

A preschedule S_0 is generated offline before execution begins, assigning a start time s_i^0 and machine assignment $M(i)$ to every operation. S_0 is assumed to be feasible and near-optimal for the static problem. During execution, the schedule $S(t)$ evolves as disruptions occur and responses are applied; S_0 serves as the permanent reference for deviation measurement throughout the planning horizon.

A disruption event is formally a tuple $E = (\psi, k, \tau, \lambda, A(E))$ where $\psi \in \{BD, PC\}$ is the disruption type (machine breakdown or processing time change), k is the disrupted machine, τ is the arrival time, λ is the duration (or processing time change magnitude for PC disruptions), and $A(E)$ is the set of operations whose scheduled start times are directly threatened. Disruption events arrive according to a Poisson process with machine-breakdown rate Λ_b and processing-change rate Λ_p ; breakdown durations follow an exponential distribution with mean μ_λ ; processing time changes follow magnitude distribution $p_i = p_i \times (1 + \rho \times U(-1, 1))$, where $\rho \sim U(0, \rho_{\text{max}})$. Preemption is not permitted; disrupted operations restart from the beginning after machine recovery.

The governor's decision problem is: at each disruption event E occurring at time t , select a response strategy s^* from the ordered action space $S = \{DN, SJM, RS, PR, RW, MU, RR, FO\}$ to minimize a composite objective that trades efficiency recovery against stability cost.

B. Notation

Table 1 defines the primary symbols used throughout the methodology. All time quantities are in abstract simulation time units. Normalized quantities are dimensionless.

Symbol	Name	Formula/V alue	Source
--------	------	----------------	--------

S_0	Preschedule	Original baseline schedule; permanent reference for all deviation measures	—
$S(t)$	Current Schedule	Executing schedule at time t after all prior governor decisions	—
$S'(s)$	Revised Schedule	Schedule resulting from applying strategy s to E	—
$M_o(S)$	Deterministic makespan	$L(o,*)$ — longest path from source to sink in $G(N,A)$	Leon et al. (1994)
sl_i	Slack of operation i	$L(o,*) - (L(o,i) + p_i + L(i,*))$	Leon et al. (1994)
$RD3(S)$	Slack surrogate	$\sum_i \in N f sl_i / Nf $	Leon et al. (1994)
$Z(S)$	GAROB surrogate	$r \times M_o(S) + (1-2r) \times RD3(S)$	Leon et al. (1994)
$D(\tau, \lambda)$	Delay function	$\max\{0, \tau + \lambda - t_i - sl_i\}$	Leon et al. (1994)
$\sigma(t)$	Governor state vector	$(S(t), RD3(t), \Delta(t), \theta(t), freq(t), util(t))$	This work
$\Delta(t)$	Accumulated deviation	$\sum_{t' \leq t} \{StabilityCost(s^*(t'), E(t'), t')\}$	This work
$\theta(t)$	Adaptive threshold	$\theta(t) \in$	This work

		$[\theta_{min}, \theta_{max}]$; updated by regret signal	
$C(s, E, t)$	Composite cost	$r \times [EL(s)/\bar{E}] + (1-r) \times [SC(s, E, t)/\bar{S}]$	This work
$U(s, E, t)$	Utility of strategy s	$C(DN, E, t) - C(s, E, t)$	Cowling & Johansson (2002)
$R(s)$	Benefit-cost ratio	$U(s, E, t) / (SC(s, E, t)/\bar{S} + \epsilon)$	This work
r	Efficiency weight	$r \in [0, 1]$; default 0.85	Leon et al. (1994)
η	Learning rate	$\eta > 0$; governs speed of θ adaptation	Online Learning Literature
Φ	Penalty cap	$M_o(S_0) / O $ — average operation duration	This work

C. Governor Architecture

The decision layer operates as a dormant monitoring process that activates on two trigger conditions: a primary trigger on disruption event detection, and a secondary trigger when the accumulated deviation $\Delta(t)$ exceeds a threshold Δ_{max} . The secondary trigger addresses the failure mode of gradual schedule degradation – where a series of individually small disruptions, each handled by right-shift repair, collectively erodes the schedule's integrity without any single event crossing the primary trigger threshold.

The governor consists of four functional components: a disruption detection and classification module, a schedule state monitor, a decision criterion module, and a strategy execution interface. These components operate sequentially on each triggering event as described below.

1) Disruption Detection and Classification

On detecting a change in machine state or operation processing time, the module constructs the disruption event tuple $E = (\psi, k, \tau, \lambda, A(E))$. The affected operation set $A(E) = \{i \in O: M(i) = k, s_i(t) \geq \tau\}$ contains all operations assigned to the disrupted machine that have not yet started. The delay function is computed for each affected operations:

$$D(\tau, \lambda) = \max \{0, \tau + \lambda - t_i - s_i\}$$

If $D(\tau, \lambda) = 0$ for all $i \in A(E)$, the disruption falls entirely within available slack and the governor outputs DN (do nothing) immediately without further evaluation. This early exit is not a simplification but a theoretically grounded result: a disruption absorbed by slack causes zero makespan impact by definition (Leon et al., 1994), so no strategy evaluation is warranted. This filter eliminates a significant fraction of minor disruptions at negligible computational cost.

2) Schedule State Monitor

The Schedule State Monitor maintains the governor's state vector $\sigma(t) = (S(t), RD3(t), \Delta(t), \theta(t), \text{freq}(t), \text{util}(t))$ continuously throughout execution. $RD3(t)$ is updated on every operation start and completion – not only on disruptions – because both events modify the disjunctive graph $G(N, A)$ and therefore change the slack profile. Operation starts reduce downstream slack by consuming machine capacity; operation completions may increase slack for operations that were waiting on the machine.

The secondary trigger fires when $\Delta(t) > \Delta_{\max}$, where $\Delta(t)$ is the running sum of all stability costs incurred by prior governor decisions since S_0 was released. In secondary trigger mode, the governor evaluates only MU and FO as candidates – partial repairs are not meaningful when the trigger accumulates damage rather than a specific new disruption.

3) Strategy Action Space

The governor selects from eight strategies ordered by ascending stability cost:

- DN – Do nothing. Returns $S(t)$ unchanged. $\text{StabilityCost} = 0$. Only valid when $D(\tau, \lambda) = 0$.
- SJM – Single Job Move. Relocates the single most-threatened operation in $A(E)$ to its cost-minimizing feasible position. Lowest non-zero stability cost.
- RS – Right-shift repair. Shifts all operations in $A(E)$ forward by λ time units, propagating changes through precedence successors. Sequence preserved.
- PR – Partial repair. Resequences $A(E)$ and all operations in $G(N, A)$ reachable from $A(E)$ using SPT dispatching. Freezes all other operations.
- RW – Rolling window repair. Resequences all operations with scheduled start times in $[\tau, \tau + W]$ using SPT dispatching, where W is a tunable window parameter.
- MU – Match-up Repair. Implements the MUSA algorithm (Bean et al., 1991): iteratively extends matchup horizon T until resequencing cost falls below EPS, then restores s_i^0 for all operations with original start times beyond T .

- RR – Rerouting repair. Reassigns operations in $A(E)$ to their best alternative eligible machine, selected by maximum available slack. Valid only in flexible job shop instances where $|M_{\text{eligible}}(i)| > 1$.
- FO – Full reoptimization. Regenerates the schedule for all remaining unexecuted operations using SPT dispatching. Maximum efficiency recovery, maximum stability cost.

D. The Decision Criterion

1) Composite Cost Function

For each candidate strategy $s \in S$, the governor computes a normalized composite cost:

$$C(s, E, t) = r \times [\text{EfficiencyLoss}(s) / \bar{E}] + (1-r) \times [\text{StabilityCost}(s) / \bar{S}]$$

Normalization is essential to the validity of the tradeoff. Without it, r controls the relative weight of two quantities with different scales and units, and the larger-scaled component dominates the sum regardless of r . Normalizing both components to $[0, 1]$ before weighting ensures r genuinely reflects the decision maker's preference between efficiency and stability. The normalizing constant $\bar{E} = M_o(S_0)$ and $\bar{S} = |A(E)| \times (\lambda + \Phi)$ are both derived from the problem instance rather than set arbitrarily.

The efficiency loss component uses Leon et al.'s (1994) GAROB surrogate:

$$\text{EfficiencyLoss}(s) = Z(S'(s)) - Z(S_0)$$

$$Z(S) = r \times M_o(S) + (1-2r) \times RD3(S)$$

$Z(S)$ captures both immediate makespan degradation and the reduction in remaining slack – penalizing strategies that produce fragile post-repair schedules that will be more vulnerable to subsequent disruptions. The use of the same parameter r in both $Z(S)$ and $C(s)$ ensures consistency: the governor's efficiency measure and its tradeoff weight reflect the same underlying preference for robustness.

The stability cost component combines Wu et al.'s (1993) start-time deviation with Rangasaritratamee et al.'s (2004) total deviation penalty:

$$\text{StabilityCost}(s, E, t) = \sum_{i \in A(E)} |s_i - s'_i(s)| + \sum_{i \in A(E)} \min \{ 10 / \sqrt{\gamma_i}, \Phi \}$$

Where $\gamma_i = s_i + s'_i(s) - 2t$ is the total deviation of operation i 's old and new start times from the current time t . The first term measures the magnitude of schedule change; the second term weights changes by urgency – rescheduling an operation that starts in minutes is penalized far more heavily than rescheduling one that starts next week. The cap $\min \{ \cdot, \Phi \}$ with $\Phi = M_o(S_0) / |O|$ prevents the penalty from becoming unbounded for operations scheduled to start almost immediately, and ensures $\text{StabilityCost}(s, E, t) / \bar{S} \in [0, 1]$.

2) Utility and Benefit-Cost Ratio

The utility of strategy s is defined as the cost reduction relative to doing nothing:

$$U(s, E, t) = C(DN, E, t) - C(s, E, t)$$

This follows the framework of Cowling and Johansson (2002), with DN serving as the baseline comparator v_0 . Any strategy with $U(s, E, t) \leq 0$ is immediately rejected – it's not worth the stability cost it imposes relative to passive absorption. The benefit-cost ratio is then:

$$R(s) = U(s, E, t) / (\text{StabilityCost}(s, E, t) / \bar{S} + \varepsilon)$$

Where $\varepsilon = 10^{-6}$ prevents division by zero when $\text{StabilityCost} = 0$. The ratio formulation, departing from Cowling and Johansson's weighted difference $A \times U - B \times S$, has two advantages: it is scale-invariant (U and StabilityCost need not be in comparable units), and it directly expresses the efficiency gain per unit of nervousness caused – the natural criterion for a conservatism-preferring governor.

3) Strategy Selection Rule

The governor selects:

$s^* = \text{least disruptive } s \in S \text{ such that } R(s) \geq \theta(t) \text{ and Feasible}(s, S(t))$

Iterating through the preference ordering $DN < SJM < RS < PR < RW < MU < RR < FO$, the first feasible strategy whose benefit-cost ratio meets or exceeds the current threshold $\theta(t)$ is selected. If no strategy meets the threshold, the governor outputs DN – the disruption is passively absorbed. This fallback behaviour means the governor conserves stability when no strategy provides sufficient benefit, rather than defaulting to aggressive reoptimization.

E. The Adaptive Threshold Mechanism

1) Threshold Initialization and Bounds

The adaptive threshold $\theta(t) \in [\theta_{\min}, \theta_{\max}]$ is initialized at the midpoints $\theta(0) = (\theta_{\min} + \theta_{\max})/2$, reflecting no prior knowledge of the disruption environment. The bounds $\theta_{\min} > 0$ and $\theta_{\max} < \infty$ prevent degenerate behaviour: $\theta_{\min}$ ensures the governor cannot collapse to always-do-nothing if negative regret repeatedly drives θ downward; $\theta_{\max}$ ensures it cannot escalate to always-reoptimize if positive regret drives θ upward.

2) Regret Computation

After executing s^* and observing the outcome over the interval $W_{\text{obs}} = [t, t_{\text{next}}]$ – where t_{next} is the next disruption arrival – the governor computes regret as:

$$\text{regret}(t) = C_{\text{observed}}(s^*(t)) - C_{\text{observed}}(s_{\text{best}}(t))$$

Where $s_{\text{best}}(t) = \text{argmin}_{\{s \in S\}} C_{\text{observed}}(s, E(t), t)$ is the strategy with the lowest observed cost determined by counterfactual simulation. In the research implementation, a complete state snapshot $\sigma_{\text{snapshot}}(t)$ is stored at disruption time t ; the simulation is branched for each of the eight

candidate strategies, each branch is run forward to t_{next} , and C_{observed} is recorded from each branch outcome. This provides a ground-truth regret signal.

The benchmark against which regret is measured is the best fixed threshold strategy in hindsight – the static θ^* that would have minimized cumulative cost over the observed disruption sequence. This is the standard benchmark in online learning theory and is practically meaningful: it measures whether the adaptive mechanism performs at least as well as the best tuned static policy, without requiring the unrealistic assumption of future knowledge.

3) Threshold Update Rule

The threshold is updated after each disruption event using a clipped gradient step:

$$\theta(t+1) = \text{clip}(\theta(t) + \eta \times \text{regret}(t), \theta_{\min}, \theta_{\max})$$

Where $\text{clip}(x, a, b) = \max\{a, \min\{x, b\}\}$ and $\eta > 0$ is the learning rate. Positive regret – the governor chose worse than the best available strategy – drives θ upward, making the governor more selective at the next event. Negative regret – the governor was unnecessarily conservative – drives θ downward. Small η produces stable but slow calibration suitable for stationary environments; large η produces fast adaptation at the risk of threshold oscillation in near-stationary environments. Sensitivity analysis varies η across $\{0.01, 0.05, 0.10, 0.20\}$.

In a stationary disruption environment, $\theta(t)$ is expected to converge toward θ^* , the optimal fixed threshold for the environment, with per-disruption regret decreasing toward zero. When the disruption environment changes – as occurs when production mix shifts or a new disruption pattern emerges – $\theta(t)$ drifts to the new optimal value without requiring manual recalibration. The formal theoretical analysis of convergence rates is identified as a direction for future work; the present paper demonstrates convergence empirically through simulation.

4) Secondary Trigger

In addition to per-disruption threshold adaptation, the governor monitors accumulated deviation $\Delta(t)$ continuously. When $\Delta(t) > \Delta_{\max}$, a secondary trigger fires independent of the new disruption event. The governor evaluates MU and FO in secondary trigger mode, potentially applying a proactive reoptimization to rebuild schedule slack before the next disruption arrives. This prevents the death-by-a-thousand-right-shifts failure mode: a schedule that has absorbed many small disruptions through sequential right-shifts may have $RD3(t) \approx 0$, meaning any future disruption will immediately cause makespan delay, but no individual disruption has been large enough to cross the primary trigger's decision threshold. The secondary trigger formalizes the decision to preemptively rebuild schedule health, a decision that purely event-driven systems cannot make.

IV. RESULT

This is just a pre-research paper, the experiment and results are yet to be done.

V. FUTURE SCOPE

REFERENCES

- [1] J. Vieira, J. Herrmann, and E. Lin, "Rescheduling manufacturing systems: A framework of strategies, policies, and methods," *Journal of Scheduling*, vol. 6, no. 1, pp. 39–62, 2003.
- [2] D. Ouelhadj and S. Petrovic, "A survey of dynamic scheduling in manufacturing systems," *Journal of Scheduling*, vol. 12, no. 4, pp. 417–431, 2009.
- [3] P. Cowling and M. Johansson, "Using real time information for effective dynamic scheduling," *European Journal of Operational Research*, vol. 139, no. 2, pp. 230–244, 2002.
- [4] R. Rangaratsamee, W. G. Ferrell, Jr., and M. B. Kurz, "Dynamic rescheduling that simultaneously considers efficiency and stability," *Computers & Industrial Engineering*, vol. 46, no. 1, pp. 1–15, 2004.
- [5] S. D. Wu, R. H. Storer, and P. C. Chang, "One-machine rescheduling heuristics with efficiency and stability as criteria," *Computers & Operations Research*, vol. 20, no. 1, pp. 1–14, 1993.
- [6] V. J. Leon, S. D. Wu, and R. H. Storer, "Robustness measures and robust scheduling for job shops," *IIE Transactions*, vol. 26, no. 5, pp. 32–43, 1994.
- [7] J. C. Bean, J. R. Birge, J. Mittenthal, and C. E. Noon, "Match-up scheduling with multiple resources, release dates, and disruptions," *Operations Research*, vol. 39, no. 3, pp. 470–483, 1991.
- [8] J. C. Larsen and M. Pranzo, "A framework for dynamic rescheduling problems," *International Journal of Production Research*, vol. 57, no. 1, pp. 16–33, 2019.